The path drawing can be handled with the `clipAngle` property of the projection, which clips any paths drawn with that projection if they fall outside of a particular angle from its center. This can be useful to show only small parts of your dataset for performance or display purposes. Here's how it looks in our new projection code:

```
projection = d3.geo.orthographic()
    .scale(200)
    .translate([width / 2, height / 2])
    .clipAngle(90);
```

This won't work for the circles we're using for our cities, because `clipAngle` only applies to data that's created by `d3.geo.path()`. For the circles, we have to ensure that they're only displayed if they fall within that clip angle. Taking this into account, we can pass a test in the zoomed function to determine whether a city should be displayed based on its coordinates.

Listing 7.14 Hiding cities on the other side of a rotated globe

```
function zoomed() {
    var currentRotate = projection.rotate()[0];
    projection.scale(mapZoom.scale());
    d3.selectAll("path.graticule").attr("d", geoPath);
    d3.selectAll("path.countries").attr("d", geoPath);

    d3.selectAll("circle.cities")
        .attr("cx", function(d) {return projection([d.x,d.y])[0]})
        .attr("cy", function(d) {return projection([d.x,d.y])[1]})
        .style("display", function(d) {
            return parseInt(d.y) + currentRotate < 90 &&
                        parseInt(d.y) + currentRotate > -90 ?
                        "block" : "none";
        });
};
```

If this city's y position is within 90 degrees of the current rotation of the globe, then display it; otherwise, hide it.

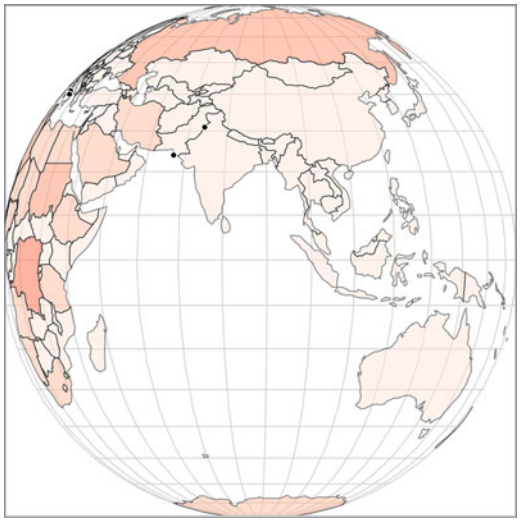


Figure 7.13 Our rotating and properly clipped globe